

# Reviewing Code Execution Sandbox

<https://gemini.google.com/app/6330d6324d42ea7c>

User prompt: I drafted a python code to server as a hermes agent sandbox. I want you to review it and then inspect if I can run python/java/C/rust codes that are on multiple files, or can I generate pdf from multiple latex file (tex + bib for instance). `` import os import sys import uuid import time import base64 import shutil import asyncio import subprocess from pathlib import Path from typing import Dict, Optional, Any from fastapi import FastAPI, HTTPException from pydantic import BaseModel app = FastAPI(title="Homelab Agent Execution Sandbox", version="1.0.0") SANDBOX\_BASE = Path(os.environ.get("SANDBOX\_DIR", "/ramdisk/sandbox")) RUNS\_DIR = SANDBOX\_BASE / "runs" BIN\_DIR = SANDBOX\_BASE / "bin" VENV\_DIR = SANDBOX\_BASE / "venv" # Ensure directories exist RUNS\_DIR.mkdir(parents=True, exist\_ok=True) BIN\_DIR.mkdir(parents=True, exist\_ok=True) class ExecuteRequest(BaseModel): language: str # python, latex, c, cpp, rust, java, sh code: str files: Optional[Dict[str, str]] = None # filename -> text or base64 content timeout: Optional[int] = 60 class ExecuteResponse(BaseModel): exit\_code: int stdout: str stderr: str duration\_ms: float artifacts: Dict[str, str] = {} # filename -> base64 or data uri def get\_python\_path() -> str: venv\_py = VENV\_DIR / "bin" / "python3" if venv\_py.exists(): return str(venv\_py) return sys.executable def get\_binary(name: str) -> str: ram\_bin = BIN\_DIR / name if ram\_bin.exists(): return str(ram\_bin) return shutil.which(name) or name @app.get("/health") def health(): return {"status": "ok", "sandbox\_dir": str(SANDBOX\_BASE)} @app.get("/runtimes") def runtimes(): info = {} # Python py\_path = get\_python\_path() try: out = subprocess.check\_output([py\_path, "--version"], text=True).strip() info["python"] = {"version": out, "path": py\_path} except Exception as e: info["python"] = {"error": str(e)} # Tectonic (LaTeX) tectonic\_path = get\_binary("tectonic") try: out = subprocess.check\_output([tectonic\_path, "--version"], text=True).splitlines()[0] info["tectonic\_latex"] = {"version": out, "path": tectonic\_path} except Exception as e: info["tectonic\_latex"] = {"error": str(e)} # GCC / C gcc\_path = get\_binary("gcc") try: out = subprocess.check\_output([gcc\_path, "--version"], text=True).splitlines()[0] info["c\_gcc"] = {"version": out, "path": gcc\_path} except Exception as e: info["c\_gcc"] = {"error": str(e)} # G++ / C++ gpp\_path = get\_binary("g++") try: out = subprocess.check\_output([gpp\_path, "--version"], text=True).splitlines()[0] info["cpp\_gpp"] = {"version": out, "path": gpp\_path} except Exception as e: info["cpp\_gpp"] = {"error": str(e)} # Rust rustc\_path = get\_binary("rustc") try: out = subprocess.check\_output([rustc\_path, "--version"], text=True).strip() info["rustc"] = {"version": out, "path": rustc\_path} except Exception as e: info["rustc"] = {"error": str(e)} # Java javac\_path = get\_binary("javac") try: out = subprocess.check\_output([javac\_path, "--version"], text=True).strip() info["java\_javac"] = {"version": out, "path": javac\_path} except Exception as e: info["java\_javac"] = {"error": str(e)} return info @app.post("/execute", response\_model=ExecuteResponse) async def execute(req: ExecuteRequest): lang = req.language.lower().strip() run\_id = f"run\_{int(time.time()\*1000)}\_{uuid.uuid4().hex[8:]}" workdir = RUNS\_DIR / run\_id workdir.mkdir(parents=True, exist\_ok=True) try: # Write any auxiliary files if req.files: for fname, fcontent in req.files.items(): target = workdir / fname target.parent.mkdir(parents=True, exist\_ok=True) if fcontent.startswith("data:") and ";base64," in fcontent: raw\_b64 = fcontent.split(";base64,")[1] target.write\_bytes(base64.b64decode(raw\_b64)) else: target.write\_text(fcontent, encoding="utf-8") cmd = [] env = os.environ.copy() env["PATH"] = f"{BIN\_DIR}:{VENV\_DIR}/bin:{env.get('PATH', '')}" if lang in ["python", "py"]: script\_path = workdir / "script.py" script\_path.write\_text(req.code, encoding="utf-8") cmd = [get\_python\_path(), "script.py"] elif lang in ["latex", "tex", "tectonic"]: tex\_path = workdir / "document.tex" tex\_path.write\_text(req.code, encoding="utf-8") cmd = [get\_binary("tectonic"), "-o", str(workdir), "document.tex"] elif lang in ["c"]: c\_path = workdir / "main.c" c\_path.write\_text(req.code, encoding="utf-8") compile\_cmd = f'{get\_binary("gcc")} -O2 main.c -o main -lm && ./main' cmd = f'"/bin/sh", "-c", compile\_cmd] elif lang in ["cpp", "c++"]: cpp\_path = workdir / "main.cpp" cpp\_path.write\_text(req.code, encoding="utf-8") compile\_cmd = f'{get\_binary("g++")} -O2 -std=c++20 main.cpp -o main -lm && ./main' cmd = f'"/bin/sh", "-c", compile\_cmd] elif lang in ["rust", "rs"]: rs\_path = workdir / "main.rs" rs\_path.write\_text(req.code, encoding="utf-8") compile\_cmd = f'{get\_binary("rustc")} -O main.rs -o main && ./main' cmd = f'"/bin/sh", "-c", compile\_cmd] elif lang in ["java"]: java\_path = workdir / "Main.java" java\_path.write\_text(req.code, encoding="utf-8") compile\_cmd = f'{get\_binary("javac")} Main.java && {get\_binary("java")} Main' cmd = f'"/bin/sh", "-c", compile\_cmd] elif lang in ["sh", "bash", "shell"]: sh\_path = workdir / "script.sh" sh\_path.write\_text(req.code, encoding="utf-8") sh\_path.chmod(0o755) cmd = f'"/bin/bash", "script.sh"] else: raise HTTPException(status\_code=400, detail=f"Unsupported language: {lang}. Supported: python, latex, c, cpp, rust, java, sh") t0 = time.time() proc = await asyncio.create\_subprocess\_exec(\*cmd, cwd=str(workdir), stdout=asyncio.subprocess.PIPE, stderr=asyncio.subprocess.PIPE, env=env) try: stdout\_b, stderr\_b = await asyncio.wait\_for(proc.communicate(), timeout=req.timeout) exit\_code = proc.returncode except asyncio.TimeoutError: proc.kill() stdout\_b, stderr\_b = b"", f"Execution timed out after {req.timeout} seconds".encode() exit\_code = -1 duration\_ms = (time.time() - t0) \* 1000 # Collect generated output artifacts (PDFs, PNGs, SVGs, JSONs, CSVs) artifacts = {} for f in workdir.glob("\*\*"): if f.is\_file() and f.suffix.lower() in [".pdf", ".png", ".jpg", ".svg", ".json", ".csv", ".html"]: # If it's a PDF if f.suffix.lower() == ".pdf": b64 = base64.b64encode(f.read\_bytes()).decode("utf-8") artifacts[f.name] = f"data:application/pdf;base64,{b64}" elif f.suffix.lower() in [".png", ".jpg", ".svg"]: mime = "image/png" if f.suffix.lower() == ".png" else "image/svg+xml" if f.suffix.lower() == ".svg" else "image/jpeg" b64 = base64.b64encode(f.read\_bytes()).decode("utf-8") artifacts[f.name] = f"data:{mime};base64,{b64}" else: try: artifacts[f.name] = f.read\_text(encoding="utf-8") except Exception: b64 = base64.b64encode(f.read\_bytes()).decode("utf-8") artifacts[f.name] = f"data:application/octet-stream;base64,{b64}" return ExecuteResponse(exit\_code=exit\_code, stdout=stdout\_b.decode("utf-8", errors="replace"), stderr=stderr\_b.decode("utf-8", errors="replace"), duration\_ms=round(duration\_ms, 2), artifacts=artifacts) finally: # Clean up ephemeral workdir shutil.rmtree(workdir, ignore\_errors=True) ``

Response: This is a solid start for a code execution sandbox. You already have a great structure in place with FastAPI, ephemeral directories, artifact collection, and asynchronous execution with timeouts.

To answer your main question: **Yes, you can run multi-file programs and LaTeX projects, but you need to tweak a few of your compilation commands.** As currently written, some languages will ignore the extra files you inject.

Here is a breakdown of how your current code handles multi-file executions, along with a critical security fix you need to make before running this.

## 1. Multi-file Execution Capabilities

Your current logic successfully writes req.files to the workdir. However, the execution commands for compiled languages are hardcoded to only look at the main file.

- **Python:**  **Works perfectly.** Because req.files are written to the workdir and script.py is executed from that same directory, any Python import statements will naturally resolve the uploaded files.
- **LaTeX (Tectonic):**  **Works perfectly.** Tectonic will automatically resolve dependencies like \bibliography{refs.bib}, \input{chapter1.tex}, or \includegraphics{image.png} as long as they are provided in req.files.
- **C / C++:**  **Fails.** Your current command is gcc -O2 main.c -o main. If a user uploads math\_helpers.c in req.files, GCC won't compile it, and the linker will fail.
  - *Fix:* Change the command to compile all .c or .cpp files in the directory.
- **Java:**  **Partially Works.** Your command is javac Main.java. The Java compiler does implicitly compile referenced classes (e.g., if Main uses Helper, it will compile Helper.java if present). However, it's safer to explicitly compile all files to prevent edge-case failures.

- *Fix:* Change it to `javac *.java`.

- **Rust:** 🚩 **Partially Works.** Your command is `rustc -O main.rs`. If `main.rs` contains `mod helper;`, `rustc` will look for `helper.rs` and compile it. However, this won't support standard Cargo projects (which require a `Cargo.toml` and `cargo run`). For single-file-entrypoint Rust, it's fine.

## 2. Critical Security Vulnerability (Path Traversal)

The way you handle `req.files` is currently vulnerable to directory traversal attacks. A malicious user could send a payload like this:

```
"files": {"../../../../../etc/passwd": "hacked"}
```

`pathlib` does not automatically stop this. `workdir / "../../../../../etc/passwd"` will resolve to the host's root file system, and your sandbox will overwrite system files.

## 3. Recommended Code Updates

Here is how you should update your `/execute` endpoint to fix the multi-file compilation commands and patch the path traversal vulnerability:

```
# 1. Update the file writing logic (Security Fix)
if req.files:
    workdir_resolved = workdir.resolve()
    for fname, fcontent in req.files.items():
        target = (workdir / fname).resolve()

        # Prevent directory traversal escaping the sandbox
        if not target.is_relative_to(workdir_resolved):
            raise HTTPException(status_code=400, detail=f"Invalid file path: {fname}")

        target.parent.mkdir(parents=True, exist_ok=True)
        if fcontent.startswith("data:") and ";base64," in fcontent:
            raw_b64 = fcontent.split(";base64,")[1]
            target.write_bytes(base64.b64decode(raw_b64))
        else:
            target.write_text(fcontent, encoding="utf-8")

# 2. Update C compilation
elif lang in ["c"]:
    c_path = workdir / "main.c"
    c_path.write_text(req.code, encoding="utf-8")
    # Change: Use *.c to compile all provided source files
    compile_cmd = f"{get_binary('gcc')} -O2 *.c -o main -lm && ./main"
    cmd = ["/bin/sh", "-c", compile_cmd]

# 3. Update C++ compilation
elif lang in ["cpp", "c++"]:
    cpp_path = workdir / "main.cpp"
    cpp_path.write_text(req.code, encoding="utf-8")
    # Change: Use *.cpp to compile all provided source files
    compile_cmd = f"{get_binary('g++')} -O2 -std=c++20 *.cpp -o main -lm && ./main"
    cmd = ["/bin/sh", "-c", compile_cmd]

# 4. Update Java compilation
elif lang in ["java"]:
    java_path = workdir / "Main.java"
    java_path.write_text(req.code, encoding="utf-8")
    # Change: Compile all *.java files explicitly
    compile_cmd = f"{get_binary('javac')} *.java && {get_binary('java')} Main"
    cmd = ["/bin/sh", "-c", compile_cmd]
```

## Additional Sandbox Advice

- **Artifact Collection Logic:** Your current logic sweeps the whole directory for `.pdf`, `.json`, etc. Be aware that if a user uploads a `.json` configuration file in `req.files`, it will be returned back to them in the `artifacts` response at the end. If you only want *generated* artifacts, you might want to snapshot the directory contents before execution and only grab *new* or *modified* files.
- **Memory/CPU Limits:** Since this is meant to be a sandbox, `subprocess` alone doesn't prevent a user from writing a `while(True)` loop that consumes 100% CPU, or a memory bomb that exhausts your RAM before the timeout hits. If you aren't running this FastAPI app inside a tightly constrained Docker container with `cgroups` (memory/CPU limits), you should do that.

---

*User prompt: I have another question, does this code accepts sessions ? Let's say that I have a project with about 50 python files. At first run, the result is not satisfactory, so I updated one file. In the second run, do I have to re-send all the files or just the updated one ?*

Response: **No, your current code does not support sessions.** It is entirely stateless by design.

If you update one file in a 50-file project and send just that updated file on the second run, the execution will fail because the other 49 files will be missing.

Here is exactly why this happens in your current code:

1. **Unique Directory per Request:** Every time you hit /execute, you generate a brand new run\_id with a UUID and timestamp.
2. **Ephemeral Destruction:** In your finally block, you call `shutil.rmtree(workdir, ignore_errors=True)`. This completely deletes the sandbox folder the moment the execution finishes or times out.

## How to Add Session Support

To support incremental updates (sending only the changed files), you need to decouple the workspace lifecycle from the execution lifecycle.

Here is how you can modify your code to support a session\_id:

### 1. Update your Data Model

Add an optional session\_id to your request payload.

```
class ExecuteRequest(BaseModel):
    language: str
    code: str
    files: Optional[Dict[str, str]] = None
    timeout: Optional[int] = 60
    session_id: Optional[str] = None # New field
```

### 2. Modify Directory Creation

Instead of always generating a random UUID, reuse the directory if a session\_id is provided. **Crucially, sanitize the session\_id** to prevent directory traversal.

```
import re

@app.post("/execute", response_model=ExecuteResponse)
async def execute(req: ExecuteRequest):
    lang = req.language.lower().strip()

    # 1. Determine Workdir based on Session
    if req.session_id:
        # Sanitize to alphanumeric and dashes only to prevent path traversal
        safe_session = re.sub(r'[^a-zA-Z0-9\-]', '', req.session_id)
        if not safe_session:
            raise HTTPException(status_code=400, detail="Invalid session_id format")
        run_id = f"session_{safe_session}"
    else:
        run_id = f"run_{int(time.time()*1000)}_{uuid.uuid4().hex[:8]}"

    workdir = RUNS_DIR / run_id
    workdir.mkdir(parents=True, exist_ok=True)

    # ... (rest of your execution and file writing logic stays exactly the same) ...
```

### 3. Conditional Cleanup

Modify your finally block so it only deletes the folder if it's a transient, session-less run.

```
finally:
    # Only clean up ephemeral runs. Session directories persist.
    if not req.session_id:
        shutil.rmtree(workdir, ignore_errors=True)
```

## The Trade-off: Ramdisk Exhaustion

Because your sandbox operates in a ramdisk (/ramdisk/sandbox), keeping session directories alive means your RAM usage will strictly increase over time. The 50-file projects, compiled binaries, and generated PDFs will sit in memory permanently until the server restarts.

If you implement sessions, you **must** implement a garbage collector.

---

*User prompt: Yes. So we need to start a session, send and resend, and then close the session ? A session could have a time to live. Please consider handling Critical Security Vulnerability . Make the new code optimized. Write you enhanced version of this python code.*

*Response: This updated version introduces stateful sessions with a background garbage collector, patches the directory traversal vulnerability, and optimizes file I/O operations by offloading them from the main asynchronous event loop.*

It also implements the multi-file compilation commands for C, C++, and Java.

## The Enhanced Sandbox Code

```
import os
import sys
import uuid
import time
```

```

import base64
import shutil
import asyncio
import re
from pathlib import Path
from typing import Dict, Optional, Any
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException, BackgroundTasks
from pydantic import BaseModel, Field

# Configuration
SANDBOX_BASE = Path(os.environ.get("SANDBOX_DIR", "/ramdisk/sandbox"))
RUNS_DIR = SANDBOX_BASE / "runs"
BIN_DIR = SANDBOX_BASE / "bin"
VENV_DIR = SANDBOX_BASE / "venv"
SESSION_TTL_SECONDS = int(os.environ.get("SESSION_TTL", 3600)) # 1 hour default

# Ensure directories exist
RUNS_DIR.mkdir(parents=True, exist_ok=True)
BIN_DIR.mkdir(parents=True, exist_ok=True)

# --- Background Task for Session TTL Cleanup ---
async def cleanup_stale_sessions():
    """Runs periodically to delete sessions that exceed their TTL."""
    while True:
        await asyncio.sleep(300) # Check every 5 minutes
        now = time.time()
        for d in RUNS_DIR.glob("session_*"):
            if d.is_dir():
                try:
                    # Check last modification time
                    if now - d.stat().st_mtime > SESSION_TTL_SECONDS:
                        shutil.rmtree(d, ignore_errors=True)
                except Exception:
                    pass

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Start the background cleanup task on startup
    task = asyncio.create_task(cleanup_stale_sessions())
    yield
    # Cancel the task on shutdown
    task.cancel()

app = FastAPI(title="Homelab Agent Execution Sandbox", version="1.1.0", lifespan=lifespan)

# --- Data Models ---
class ExecuteRequest(BaseModel):
    language: str
    code: str
    files: Optional[Dict[str, str]] = None
    timeout: Optional[int] = Field(default=60, le=300) # Max 5 min timeout
    session_id: Optional[str] = Field(default=None, pattern=r"^[a-zA-Z0-9_-]+$") # Sanitize input

class ExecuteResponse(BaseModel):
    exit_code: int
    stdout: str
    stderr: str
    duration_ms: float
    session_id: Optional[str]
    artifacts: Dict[str, str] = {}

# --- Utilities ---
def get_python_path() -> str:
    venv_py = VENV_DIR / "bin" / "python3"
    return str(venv_py) if venv_py.exists() else sys.executable

def get_binary(name: str) -> str:
    ram_bin = BIN_DIR / name
    return str(ram_bin) if ram_bin.exists() else (shutil.which(name) or name)

def process_file_writes(workdir: Path, code: str, entry_filename: str, files: Optional[Dict[str, str]]):
    """Handles file I/O synchronously. Designed to be run in a thread pool."""
    workdir_resolved = workdir.resolve()

    # 1. Write auxiliary files securely
    if files:
        for fname, fcontent in files.items():
            target = (workdir / fname).resolve()

```

```

# CRITICAL SECURITY: Prevent Directory Traversal
if not target.is_relative_to(workdir_resolved):
    raise ValueError(f"Path traversal blocked for file: {fname}")

target.parent.mkdir(parents=True, exist_ok=True)
if fcontent.startswith("data:") and ";base64," in fcontent:
    raw_b64 = fcontent.split(";base64,")[1]
    target.write_bytes(base64.b64decode(raw_b64))
else:
    target.write_text(fcontent, encoding="utf-8")

# 2. Write the main code payload to the entrypoint file
entry_path = workdir / entry_filename
entry_path.write_text(code, encoding="utf-8")

# 3. Touch the directory to update st_mtime for TTL tracking
workdir.touch()

def collect_artifacts(workdir: Path) -> Dict[str, str]:
    """Collects generated artifacts. Runs in a thread pool."""
    artifacts = {}
    for f in workdir.glob("**/*"): # Support nested directories
        if f.is_file() and f.suffix.lower() in [".pdf", ".png", ".jpg", ".svg", ".json", ".csv", ".html"]:
            suffix = f.suffix.lower()
            if suffix == ".pdf":
                mime = "application/pdf"
            elif suffix == ".png":
                mime = "image/png"
            elif suffix == ".jpg":
                mime = "image/jpeg"
            elif suffix == ".svg":
                mime = "image/svg+xml"
            else:
                try:
                    artifacts[f.name] = f.read_text(encoding="utf-8")
                    continue
                except Exception:
                    mime = "application/octet-stream"

            b64 = base64.b64encode(f.read_bytes()).decode("utf-8")
            artifacts[f.name] = f"data:{mime};base64,{b64}"
    return artifacts

# --- Endpoints ---
@app.delete("/sessions/{session_id}")
async def close_session(session_id: str):
    """Manually close and purge a session before TTL expires."""
    safe_session = re.sub(r'^a-zA-Z0-9_-]', '', session_id)
    workdir = RUNS_DIR / f"session_{safe_session}"
    if workdir.exists():
        await asyncio.to_thread(shutil.rmtree, workdir, ignore_errors=True)
    return {"status": "closed", "session_id": safe_session}
    raise HTTPException(status_code=404, detail="Session not found")

@app.post("/execute", response_model=ExecuteResponse)
async def execute(req: ExecuteRequest):
    lang = req.language.lower().strip()

    # Session routing
    is_ephemeral = not req.session_id
    if is_ephemeral:
        current_run_id = f"run_{int(time.time()*1000)}_{uuid.uuid4().hex[:8]}"
        workdir = RUNS_DIR / current_run_id
    else:
        current_run_id = req.session_id
        workdir = RUNS_DIR / f"session_{req.session_id}"

    workdir.mkdir(parents=True, exist_ok=True)

    try:
        # Determine execution strategy
        env = os.environ.copy()
        env["PATH"] = f"{BIN_DIR}:{VENV_DIR}/bin:{env.get('PATH', '')}"

        if lang in ["python", "py"]:
            entry_file = "script.py"
            cmd = [get_python_path(), "script.py"]
        elif lang in ["latex", "tex", "tectonic"]:

```

```

        entry_file = "document.tex"
        cmd = [get_binary("tectonic"), "-o", str(workdir), "document.tex"]
    elif lang in ["c"]:
        entry_file = "main.c"
        compile_cmd = f"{get_binary('gcc')} -O2 *.c -o main -lm && ./main"
        cmd = ["/bin/sh", "-c", compile_cmd]
    elif lang in ["cpp", "c++"]:
        entry_file = "main.cpp"
        compile_cmd = f"{get_binary('g++')} -O2 -std=c++20 *.cpp -o main -lm && ./main"
        cmd = ["/bin/sh", "-c", compile_cmd]
    elif lang in ["rust", "rs"]:
        entry_file = "main.rs"
        compile_cmd = f"{get_binary('rustc')} -O main.rs -o main && ./main"
        cmd = ["/bin/sh", "-c", compile_cmd]
    elif lang in ["java"]:
        entry_file = "Main.java"
        compile_cmd = f"{get_binary('javac')} *.java && {get_binary('java')} Main"
        cmd = ["/bin/sh", "-c", compile_cmd]
    elif lang in ["sh", "bash", "shell"]:
        entry_file = "script.sh"
        cmd = ["/bin/bash", "script.sh"]
    else:
        raise HTTPException(status_code=400, detail=f"Unsupported language: {lang}")

    # Execute file I/O safely in a background thread to unblock FastAPI
    try:
        await asyncio.to_thread(process_file_writes, workdir, req.code, entry_file, req.files)
    except ValueError as e:
        raise HTTPException(status_code=403, detail=str(e))

    if lang in ["sh", "bash", "shell"]:
        (workdir / entry_file).chmod(0o755)

    # Execute Subprocess
    t0 = time.time()
    proc = await asyncio.create_subprocess_exec(
        *cmd,
        cwd=str(workdir),
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
        env=env
    )

    try:
        stdout_b, stderr_b = await asyncio.wait_for(proc.communicate(), timeout=req.timeout)
        exit_code = proc.returncode
    except asyncio.TimeoutError:
        proc.kill()
        stdout_b, stderr_b = b"", f"Execution timed out after {req.timeout} seconds".encode()
        exit_code = -1

    duration_ms = (time.time() - t0) * 1000

    # Collect artifacts in a background thread
    artifacts = await asyncio.to_thread(collect_artifacts, workdir)

    return ExecuteResponse(
        exit_code=exit_code,
        stdout=stdout_b.decode("utf-8", errors="replace"),
        stderr=stderr_b.decode("utf-8", errors="replace"),
        duration_ms=round(duration_ms, 2),
        session_id=req.session_id,
        artifacts=artifacts
    )

finally:
    # Only purge the directory immediately if it's an ephemeral run without a session
    if is_ephemeral:
        await asyncio.to_thread(shutil.rmtree, workdir, ignore_errors=True)

```

## Key Architectural Upgrades

- **FastAPI Lifespan Context:** The `cleanup_stale_sessions` function runs continuously as a background task tied to the application's lifecycle. It wakes up every 5 minutes and checks the `st_mtime` (last modified time) of all session directories, purging anything older than 1 hour.
- **Asynchronous I/O Offloading:** File writing and artifact reading are now wrapped in `asyncio.to_thread()`. Under heavy load, standard Python `pathlib` synchronous writes will block the main ASGI event loop. Pushing this to a thread pool keeps the FastAPI server responsive while processing large file trees.

- **Path Traversal Prevention:** The `process_file_writes` utility resolves both the target path and the base sandbox path dynamically. It strictly enforces `is_relative_to(workdir_resolved)`. Pydantic's regex validation on the `session_id` (`^[a-zA-Z0-9_-]+$`) provides a second layer of defense against directory escape attempts in the session name itself.
- **Targeted Session Closure Endpoint:** The `DELETE /sessions/{session_id}` route allows the agent or client orchestrator to cleanly free memory and disk space immediately after finishing a multi-step task, rather than waiting for the TTL to expire.

User prompt: Could we define this code as an MCP ?

Response: Yes, absolutely. This code is a perfect candidate to become an **MCP (Model Context Protocol) server**.

In fact, code execution sandboxes are one of the most common and powerful use cases for MCP, as they give an AI agent the ability to write, run, debug, and iterate on code in a secure environment.

Currently, your code is structured as a standard **REST API** using FastAPI. To define it as an MCP server, you need to change the communication protocol from HTTP REST to **JSON-RPC** (which MCP uses over either `stdio` for local execution or SSE for remote execution).

Here is how your current architecture maps directly to MCP concepts, followed by how you would rewrite it using the official Python MCP SDK.

## Mapping Your API to MCP

| FastAPI Concept     | MCP Concept       | How it works in MCP                                                                                                                                                                                                      |
|---------------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST /execute       | Tool              | You would define an <code>execute_code</code> tool. The agent calls this tool with arguments ( <code>language</code> , <code>code</code> , <code>files</code> , <code>session_id</code> ).                               |
| Generated Artifacts | Resources         | Instead of sending large base64 PDFs back in the tool response, you can expose generated files as MCP Resources (e.g., <code>sandbox://{session_id}/output.pdf</code> ), allowing the agent to read them only if needed. |
| GET /runtimes       | Prompts / Context | You can inject the available runtimes into the agent's context dynamically so it knows exactly which versions of GCC, Python, or Tectonic are available before it writes the code.                                       |

## How to Convert It Using FastMCP

The official Python MCP library includes a wrapper called `FastMCP` which is heavily inspired by FastAPI. If you were to port your current logic to MCP, it would look remarkably similar.

Here is a conceptual example of how your sandbox translates into an MCP server over `stdio`:

```
from typing import Dict, Optional
from mcp.server.fastmcp import FastMCP
import asyncio

# Initialize the MCP Server
mcp = FastMCP("Homelab Execution Sandbox")

# 1. Define the Execution Tool
@mcp.tool()
async def execute_code(
    language: str,
    code: str,
    files: Optional[Dict[str, str]] = None,
    session_id: Optional[str] = None
) -> str:
    """
    Executes Python, C, C++, Rust, Java, or LaTeX code in a secure sandbox.
    Use session_id to maintain state and files across multiple executions.
    """

    # ... (Your existing logic for handling sessions, writing files, and subprocess execution goes here) .

    stdout = "Hello World"
    exit_code = 0

    if exit_code == 0:
        return f"Execution Successful.\nStdout: {stdout}"
    else:
        return f"Execution Failed (Code {exit_code}).\nStderr: {stderr}"

# 2. Define a Tool to Close Sessions
@mcp.tool()
async def close_session(session_id: str) -> str:
    """Closes an active sandbox session and frees memory."""
    # ... (Your existing cleanup logic) ...
```

```

    return f"Session {session_id} closed."

# 3. Expose Artifacts as Resources
@mcp.resource("sandbox://{session_id}/{filename}")
def get_artifact(session_id: str, filename: str) -> str:
    """Read a generated artifact like a CSV or log file from the sandbox."""
    # ... (Logic to read the file from the session's workdir) ...
    return file_content

if __name__ == "__main__":
    # This starts the MCP server listening on standard input/output
    # Your agent (like Hermes) can now spawn this process and communicate with it
    mcp.run()

```

## Why switch to MCP?

- 1. Native Agent Integration:** If your Hermes agent (or other tools like Claude Desktop or Open-WebUI) supports MCP, you don't need to write custom API-calling logic for the agent. You just point the agent at the MCP server, and it automatically understands how to use the `execute_code` tool.
- 2. Context Management:** Instead of returning massive base64 strings in the HTTP response (which can blow up the agent's context window), MCP allows you to return text summaries (e.g., "PDF generated successfully") and let the agent decide if it wants to load the actual image/PDF using an MCP Resource request.

---

*User prompt: Please rewrite my entire enhanced FastAPI code into a fully functional MCP server using the official Python MCP SDK.*

```

Response: import os
import sys
import uuid
import time
import shutil
import asyncio
import base64
import re
import threading
from pathlib import Path

from mcp.server.fastmcp import FastMCP, Context

# --- Configuration ---
SANDBOX_BASE = Path(os.environ.get("SANDBOX_DIR", "/ramdisk/sandbox"))
RUNS_DIR = SANDBOX_BASE / "runs"
BIN_DIR = SANDBOX_BASE / "bin"
VENV_DIR = SANDBOX_BASE / "venv"
SESSION_TTL_SECONDS = int(os.environ.get("SESSION_TTL", 3600)) # 1 hour default

# Ensure directories exist
RUNS_DIR.mkdir(parents=True, exist_ok=True)
BIN_DIR.mkdir(parents=True, exist_ok=True)

# --- Background Garbage Collector ---
def cleanup_stale_sessions():
    """Runs continuously in a background thread to purge expired sessions."""
    while True:
        time.sleep(300) # Check every 5 minutes
        now = time.time()
        for d in RUNS_DIR.glob("session_*"):
            if d.is_dir():
                try:
                    if now - d.stat().st_mtime > SESSION_TTL_SECONDS:
                        shutil.rmtree(d, ignore_errors=True)
                except Exception:
                    pass

# Start the garbage collector as a daemon thread
threading.Thread(target=cleanup_stale_sessions, daemon=True).start()

# --- Initialize MCP Server ---
mcp = FastMCP("Homelab Execution Sandbox")

# --- Utilities ---
def get_python_path() -> str:
    venv_py = VENV_DIR / "bin" / "python3"
    return str(venv_py) if venv_py.exists() else sys.executable

def get_binary(name: str) -> str:

```



```

ram_bin = BIN_DIR / name
return str(ram_bin) if ram_bin.exists() else (shutil.which(name) or name)

def process_file_writes(workdir: Path, code: str, entry_filename: str, files: dict[str, str] | None):
    """Safely handles file writing to disk."""
    workdir_resolved = workdir.resolve()

    if files:
        for fname, fcontent in files.items():
            target = (workdir / fname).resolve()

            # Security: Prevent Directory Traversal
            if not target.is_relative_to(workdir_resolved):
                raise ValueError(f"Path traversal blocked for file: {fname}")

            target.parent.mkdir(parents=True, exist_ok=True)

            # Handle base64 encoded data URIs if provided
            if fcontent.startswith("data:") and ";base64," in fcontent:
                raw_b64 = fcontent.split(";base64,")[1]
                target.write_bytes(base64.b64decode(raw_b64))
            else:
                target.write_text(fcontent, encoding="utf-8")

        # Write the main entrypoint file
        entry_path = workdir / entry_filename
        entry_path.write_text(code, encoding="utf-8")

        # Update directory timestamp for TTL tracking
        workdir.touch()

# --- MCP Tools ---

@mcp.tool()
async def get_runtimes() -> str:
    """Check the availability and versions of supported compilers and runtimes."""
    info = []

    def check_cmd(cmd: list[str], name: str):
        try:
            import subprocess
            out = subprocess.check_output(cmd, text=True, stderr=subprocess.STDOUT).splitlines()[0]
            info.append(f"-- {name}*: {out.strip()}")
        except Exception as e:
            info.append(f"-- {name}*: Not found or error ({str(e)})")

    check_cmd([get_python_path(), "--version"], "Python")
    check_cmd([get_binary("tectonic"), "--version"], "LaTeX (Tectonic)")
    check_cmd([get_binary("gcc"), "--version"], "C (GCC)")
    check_cmd([get_binary("g++"), "--version"], "C++ (G++)")
    check_cmd([get_binary("rustc"), "--version"], "Rust (rustc)")
    check_cmd([get_binary("javac"), "--version"], "Java (javac)")

    return "### Available Runtimes\n" + "\n".join(info)

@mcp.tool()
async def execute_code(
    language: str,
    code: str,
    files: dict[str, str] | None = None,
    timeout: int = 60,
    session_id: str | None = None
) -> str:
    """
    Executes code in a secure sandbox.

    Args:
        language: "python", "latex", "c", "cpp", "rust", "java", or "sh"
        code: The main source code to execute.
        files: A dictionary of {filename: content} for additional files in multi-file projects.
        timeout: Execution timeout in seconds (max 300).
        session_id: Optional ID to persist files across multiple executions.
    """
    lang = language.lower().strip()
    timeout = min(timeout, 300) # Cap at 5 minutes

    # Session setup
    is_ephemeral = not session_id
    if is_ephemeral:

```

```

current_run_id = f"run_{int(time.time()*1000)}_{uuid.uuid4().hex[:8]}"
workdir = RUNS_DIR / current_run_id
else:
    # Sanitize session_id input
    safe_session = re.sub(r'[^a-zA-Z0-9_-]', '', session_id)
    if not safe_session:
        return "Error: Invalid session_id format. Use alphanumeric and dashes."
    current_run_id = safe_session
    workdir = RUNS_DIR / f"session_{safe_session}"

workdir.mkdir(parents=True, exist_ok=True)

try:
    env = os.environ.copy()
    env["PATH"] = f"{BIN_DIR}:{VENV_DIR}/bin:{env.get('PATH', '')}"

    # Determine compilation/execution commands
    if lang in ["python", "py"]:
        entry_file = "script.py"
        cmd = [get_python_path(), "script.py"]
    elif lang in ["latex", "tex", "tectonic"]:
        entry_file = "document.tex"
        cmd = [get_binary("tectonic"), "-o", str(workdir), "document.tex"]
    elif lang in ["c"]:
        entry_file = "main.c"
        compile_cmd = f"{get_binary('gcc')} -O2 *.c -o main -lm && ./main"
        cmd = ["/bin/sh", "-c", compile_cmd]
    elif lang in ["cpp", "c++"]:
        entry_file = "main.cpp"
        compile_cmd = f"{get_binary('g++')} -O2 -std=c++20 *.cpp -o main -lm && ./main"
        cmd = ["/bin/sh", "-c", compile_cmd]
    elif lang in ["rust", "rs"]:
        entry_file = "main.rs"
        compile_cmd = f"{get_binary('rustc')} -O main.rs -o main && ./main"
        cmd = ["/bin/sh", "-c", compile_cmd]
    elif lang in ["java"]:
        entry_file = "Main.java"
        compile_cmd = f"{get_binary('javac')} *.java && {get_binary('java')} Main"
        cmd = ["/bin/sh", "-c", compile_cmd]
    elif lang in ["sh", "bash", "shell"]:
        entry_file = "script.sh"
        cmd = ["/bin/bash", "script.sh"]
    else:
        return f"Error: Unsupported language '{lang}'"

    # Offload file I/O to avoid blocking the event loop
    try:
        await asyncio.to_thread(process_file_writes, workdir, code, entry_file, files)
    except ValueError as e:
        return f"Security Error: {str(e)}"

    if lang in ["sh", "bash", "shell"]:
        (workdir / entry_file).chmod(0o755)

    # Execute
    t0 = time.time()
    proc = await asyncio.create_subprocess_exec(
        *cmd,
        cwd=str(workdir),
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
        env=env
    )

    try:
        stdout_b, stderr_b = await asyncio.wait_for(proc.communicate(), timeout=timeout)
        exit_code = proc.returncode
    except asyncio.TimeoutError:
        proc.kill()
        stdout_b, stderr_b = b"", f"Execution timed out after {timeout} seconds".encode()
        exit_code = -1

    duration_ms = (time.time() - t0) * 1000

    # Scan for generated files
    artifacts = []
    for f in workdir.glob("**/*"):
        if f.is_file() and f.name not in [entry_file, "main"]:
            artifacts.append(str(f.relative_to(workdir)))

```

```

# Construct Agent-Friendly Response
response = [
    f"### Execution Complete ({duration_ms:.0f}ms)",
    f"**Exit Code:** {exit_code}",
    f"**Session ID:** `{current_run_id}`" if not is_ephemeral else "**Run:** Ephemeral (Not saved)"
]

if stdout_b:
    response.append(f"\n**Stdout:**\n```\n{stdout_b.decode('utf-8', errors='replace')}\n```")
if stderr_b:
    response.append(f"\n**Stderr:**\n```\n{stderr_b.decode('utf-8', errors='replace')}\n```")

if not is_ephemeral and artifacts:
    response.append("\n**Files in Workspace:**")
    for art in artifacts:
        response.append(f"- `{art}` (Read using Resource URI: `sandbox://{current_run_id}/{art}`)")

return "\n".join(response)

finally:
    if is_ephemeral:
        await asyncio.to_thread(shutil.rmtree, workdir, ignore_errors=True)

@mcp.tool()
async def close_session(session_id: str) -> str:
    """Manually close and purge a session to free memory before TTL expires."""
    safe_session = re.sub(r'^[a-zA-Z0-9_-]', '', session_id)
    workdir = RUNS_DIR / f"session_{safe_session}"

    if workdir.exists():
        await asyncio.to_thread(shutil.rmtree, workdir, ignore_errors=True)
        return f"Session `{safe_session}` closed and purged."
    return f"Session `{safe_session}` not found."

# --- MCP Resources ---
@mcp.resource("sandbox://{session_id}/{filename}")
def read_sandbox_file(session_id: str, filename: str) -> str | bytes:
    """
    Read the contents of a generated file from an active session.
    Text files return strings, binary files (like PDFs or images) return bytes.
    """
    safe_session = re.sub(r'^[a-zA-Z0-9_-]', '', session_id)
    workdir = RUNS_DIR / f"session_{safe_session}"

    if not workdir.exists():
        raise ValueError("Session not found or expired.")

    target = (workdir / filename).resolve()

    if not target.is_relative_to(workdir.resolve()):
        raise ValueError("Path traversal blocked.")

    if not target.exists() or not target.is_file():
        raise ValueError("File not found.")

    # Return binary types for non-text files
    ext = target.suffix.lower()
    if ext in [".pdf", ".png", ".jpg", ".jpeg", ".zip", ".tar", ".gz"]:
        return target.read_bytes()

    # Default to text
    return target.read_text(encoding="utf-8", errors="replace")

if __name__ == "__main__":
    # Start the MCP server using standard input/output
    mcp.run()

```

User prompt: Please one question : if I already started a session, and I sent all the project files, then I updated one file, so I need to ask to re-run while sending only the updated file ?

Response: **Yes, exactly.** You only need to send the files that changed.

Because you are using a `session_id`, the sandbox directory persists on disk (in your ramdisk) between executions. Here is exactly how the flow works with the new code:

## How it works in practice:

### Run 1: The Initial Upload

- **You send:** `session_id="proj_123", code="..."` (your main file), and `files={"helper.py": "...", "config.json": "..."}.`
- **What happens:** The sandbox creates `session_proj_123`, writes all three files, and executes.

### Run 2: The Incremental Update

- **You send:** `session_id="proj_123", code="..."` (your main file, which you must always send), and `files={"helper.py": "<updated content>"}`.
- **What happens:** The sandbox reuses `session_proj_123`. It overwrites `helper.py` with the new content, overwrites the main file with the code payload, but **leaves `config.json` completely untouched**. Then, it executes again.

## One important rule to remember:

The code parameter (which becomes `script.py`, `main.c`, `document.tex`, etc., depending on the language) is rewritten on **every single execution**. Therefore, you must always provide the main entry-point code in the code parameter, even if you are only trying to update one of the auxiliary files in the files dictionary.

---

*User prompt: How can I add a new MCP tool that allows the agent to delete specific files from an active session, rather than just overwriting them?*

Response: To add a tool that deletes specific files from an active session, you can define a new `@mcp.tool()` function.

This new tool needs the same rigorous **path traversal protections** as your file writer, ensuring an agent cannot accidentally (or maliciously) delete files outside of the sandbox directory.

You can paste this code right below your existing `close_session` tool in your MCP server file:

```
@mcp.tool()
async def delete_file(session_id: str, filename: str) -> str:
    """
    Deletes a specific file or directory from an active session.

    Args:
        session_id: The ID of the active session.
        filename: The name or relative path of the file/folder to delete.
    """
    safe_session = re.sub(r'^[a-zA-Z0-9_-]', '', session_id)
    workdir = RUNS_DIR / f"session_{safe_session}"

    if not workdir.exists():
        return f"Error: Session `{safe_session}` not found or expired."

    target = (workdir / filename).resolve()
    workdir_resolved = workdir.resolve()

    # CRITICAL SECURITY: Prevent Directory Traversal (e.g., passing ".././etc/passwd")
    if not target.is_relative_to(workdir_resolved):
        return f"Security Error: Path traversal blocked for `{filename}`."

    # Prevent the agent from deleting the session root directory itself
    if target == workdir_resolved:
        return "Error: Cannot delete the root session directory. Use `close_session` instead."

    if not target.exists():
        return f"Error: Path `{filename}` does not exist in the session."

    # Define a synchronous deletion helper to run in a thread
    def _delete_target():
        if target.is_dir():
            shutil.rmtree(target)
        else:
            target.unlink()

    try:
        # Offload the blocking disk I/O to a background thread
        await asyncio.to_thread(_delete_target)
        return f"Successfully deleted `{filename}` from session `{safe_session}`."
    except Exception as e:
        return f"Error deleting `{filename}`: {str(e)}"
```

## How this works for the Agent

Now, if you have a 50-file Python project and the agent decides it wants to completely remove `legacy_helper.py` instead of overwriting it, the agent can call `delete_file(session_id="proj_123", filename="legacy_helper.py")`.

It intelligently handles both single files (`target.unlink()`) and entire directories (`shutil.rmtree()`) so the agent can clean up generated artifact folders if they are no longer needed.

---

*User prompt: Great. I have another question: sometimes, I want to test a flask python project, so : 1. can i ask to install packages using requirements.txt ? 2, if the project open a port (web port for instance), how to deal with this ?*

Response: Yes. Handling a web framework like Flask introduces two specific challenges to a sandbox environment:

1. **Dependency Isolation:** If every session installs its `requirements.txt` into the same global sandbox virtual environment, you will quickly run into version conflicts.
2. **Blocking Execution:** Flask runs an infinite loop (`app.run()`). If you use your standard `execute_code` tool, it will hang indefinitely until your 5-minute timeout kills it, meaning you can never actually browse the site.

To fix this, we need to add **Session-Specific Virtual Environments** and a **Background Process Manager**.

Here are the code additions you need to make to your MCP server.

## 1. Update the Python Path Logic

First, modify your `get_python_path` utility so it checks if the active session has its own `.venv` before falling back to the global one.

```
def get_python_path(workdir: Path) -> str:
    """Prioritizes a session-specific venv, falls back to the global one."""
    session_py = workdir / ".venv" / "bin" / "python3"
    if session_py.exists():
        return str(session_py)

    global_py = VENV_DIR / "bin" / "python3"
    return str(global_py) if global_py.exists() else sys.executable
```

## 2. Add a run\_command Tool (for requirements.txt)

Instead of hardcoding pip installs, give the agent a general-purpose command runner. This allows the agent to create a venv, install packages, or even run database migrations.

```
@mcp.tool()
async def run_command(session_id: str, command: str) -> str:
    """
    Run an arbitrary shell command in a session (e.g., pip install, ls, mkdir).
    Great for setting up environments before executing code.
    """
    safe_session = re.sub(r'^[a-zA-Z0-9_-]', '', session_id)
    workdir = RUNS_DIR / f"session_{safe_session}"

    if not workdir.exists():
        return f"Error: Session `{safe_session}` not found."

    # Execute the shell command
    proc = await asyncio.create_subprocess_shell(
        command,
        cwd=str(workdir),
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE
    )

    try:
        stdout_b, stderr_b = await asyncio.wait_for(proc.communicate(), timeout=120)
        out = stdout_b.decode('utf-8', errors='replace')
        err = stderr_b.decode('utf-8', errors='replace')
        return f"Exit Code: {proc.returncode}\nStdout:\n{out}\nStderr:\n{err}"
    except asyncio.TimeoutError:
        proc.kill()
        return "Error: Command timed out after 120 seconds."
```

## 3. Add a Background Server Manager

We need a way to track processes that aren't meant to finish. Add a global dictionary at the top of your file to store active PIDs:

```
# Global state to track background processes: {session_id: subprocess_object}
ACTIVE_SERVERS = {}
```

Then, add these two new tools to let the agent start and stop web servers.

```
@mcp.tool()
async def start_web_server(session_id: str, command: str, port: int) -> str:
```

```

"""
Starts a long-running web server (like Flask or Node) in the background.
The agent MUST write code that binds to 0.0.0.0 (not 127.0.0.1).
"""
safe_session = re.sub(r'^a-zA-Z0-9_-', '', session_id)
workdir = RUNS_DIR / f"session_{safe_session}"

if not workdir.exists():
    return "Error: Session not found."

# Stop any existing server for this session to free the port
if safe_session in ACTIVE_SERVERS:
    ACTIVE_SERVERS[safe_session].kill()
    del ACTIVE_SERVERS[safe_session]

env = os.environ.copy()

# Start the process without waiting for it to finish
proc = await asyncio.create_subprocess_shell(
    command,
    cwd=str(workdir),
    stdout=asyncio.subprocess.DEVNULL, # Ignore logs to prevent memory leaks
    stderr=asyncio.subprocess.DEVNULL,
    env=env
)

ACTIVE_SERVERS[safe_session] = proc

return (f"Background server started on port {port}. "
        f"Access it via http://<YOUR_HOST_IP>:{port}")

@mcp.tool()
async def stop_web_server(session_id: str) -> str:
    """Stops the background web server for a given session."""
    safe_session = re.sub(r'^a-zA-Z0-9_-', '', session_id)
    if safe_session in ACTIVE_SERVERS:
        ACTIVE_SERVERS[safe_session].kill()
        del ACTIVE_SERVERS[safe_session]
        return f"Server for session `{safe_session}` has been stopped."
    return "No active server found for this session."

```

*Note: You will also need to update your existing `close_session` tool and your background garbage collector to check if `safe_session` in `ACTIVE_SERVERS` and kill them when a session is purged.*

### How the Agent will use this flow for Flask:

1. **Agent writes the files:** Generates `app.py` (ensuring it uses `app.run(host="0.0.0.0", port=8080)`) and `requirements.txt` using `execute_code`.
2. **Agent builds the environment:** Calls `run_command` with `python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt`.
3. **Agent starts the app:** Calls `start_web_server` with `.venv/bin/python app.py` and `port=8080`.

*User prompt: I am using traefik. What is easier Caddy or traefik ?*

**Response:** For this specific sandbox use case—where the agent is spinning up raw background processes rather than isolated containers—**Caddy is significantly easier to automate.**

However, since you already use Traefik, you can absolutely make it work. Here is how they compare for this exact scenario.

### Traefik: Best for Containers, Verbose for Processes

Traefik's superpower is auto-discovery. When you deploy services via Docker or Podman, Traefik reads the container labels and configures itself instantly.

Because our sandbox launches raw Python subprocesses instead of containers, Traefik's auto-discovery is bypassed. To automate Traefik here, your agent would need to use Traefik's **File Provider**.

- **The Workflow:** The agent creates a specific YAML file in a directory Traefik is watching.
- **The Downside:** Traefik's configuration syntax is verbose. The agent has to correctly define a router, link it to an `entryPoint`, and map it to a `service` just to route a single port.

### Caddy: Best for REST API & Simplicity

Caddy is generally considered easier for programmatic routing because it was built with an exhaustive JSON REST API and a highly readable config file.

- **The Workflow (API):** The agent simply makes an HTTP POST request to Caddy's admin endpoint (usually `localhost:2019`), injecting the new route into memory instantly without touching a single file.
- **The Workflow (File):** If you prefer files, the agent just appends two lines to a Caddyfile (`sandbox-123.homelab.local { reverse_proxy 127.0.0.1:8080 }`) and issues a reload command.

## The Verdict

If you plan to eventually transition this sandbox so that every session runs inside its own isolated Podman or Docker container, **stick with Traefik**. The agent can just pass labels in the `docker run` command, and Traefik will handle the rest beautifully.

If you are keeping the architecture we just built (raw subprocesses on the host/ramdisk), **Caddy is much easier** to control programmatically via its REST API.

*User prompt: Ok. So better to keep my traefik as it, then you create a new caddy docker image (named homelab-sandbox-caddy) alongside the MCP server (named homelab-sandbox-mcp), then traefik will handle the caddy server port, and the mcp will configure live the caddy config to add or remove project ports.*

*Response: This is a brilliant architectural choice. By putting Caddy behind Traefik, you get the best of both worlds: Traefik continues to manage your primary homelab ingress and SSL certificates seamlessly, while Caddy acts purely as a lightweight, internal API-driven router for the ephemeral sandbox ports.*

Here is exactly how to set up this "double proxy" architecture.

## 1. The Docker Compose Architecture

Your Traefik instance will route all wildcard sandbox traffic (e.g., `*.sandbox.yourdomain.com`) to the Caddy container. Caddy will then look at the subdomain (the `session_id`) and reverse proxy it directly into the exact port running inside the MCP container.

```
services:
  homelab-sandbox-caddy:
    image: caddy:latest
    container_name: homelab-sandbox-caddy
    restart: unless-stopped
    # No ports exposed to the host! Traefik handles ingress.
    networks:
      - proxy # The network Traefik uses
      - sandbox_internal
    labels:
      - "traefik.enable=true"
      # Route any subdomain ending in .sandbox.yourdomain.com to Caddy
      - "traefik.http.routers.sandbox_caddy.rule=HostRegexp(`{subdomain:[a-zA-Z0-9-]+}.sandbox.yourdomain`)"
      - "traefik.http.services.sandbox_caddy.loadbalancer.server.port=80"
      # (Add your standard Traefik TLS/resolver labels here)

  homelab-sandbox-mcp:
    build: . # You'll need a custom Dockerfile to install Python/GCC/Rust etc.
    container_name: homelab-sandbox-mcp
    restart: unless-stopped
    privileged: true # Often needed if you want to use ramdisks/tmpfs inside securely
    tmpfs:
      - /ramdisk/sandbox:exec,mode=777
    environment:
      - SANDBOX_DIR=/ramdisk/sandbox
      - SANDBOX_DOMAIN=sandbox.yourdomain.com
    networks:
      - sandbox_internal

networks:
  proxy:
    external: true
  sandbox_internal:
    driver: bridge
```

## 2. Updating the MCP Server Code

Because Caddy's REST API accepts raw Caddyfiles natively, we do not need to mess with complex JSON updates. The MCP server can simply maintain a dictionary of active sessions, reconstruct a 10-line Caddyfile in memory, and push it to Caddy's `/load` endpoint every time a server starts or stops.

First, add `httpx` to your Python imports, and update your global state tracker:

```
import httpx
# Global state to track background processes AND their assigned ports
# Format: { "session_id": {"process": subprocess_object, "port": int} }
ACTIVE_SERVERS = {}

CADDY_API_URL = "http://homelab-sandbox-caddy:2019/load"
BASE_DOMAIN = os.environ.get("SANDBOX_DOMAIN", "sandbox.local")
```

Next, add this utility function to dynamically update Caddy:

```
async def update_caddy_routes():
    """Rebuilds the Caddyfile in memory and pushes it instantly via the API."""
    caddyfile = ""

    for session_id, srv_info in ACTIVE_SERVERS.items():
        port = srv_info['port']
        # Map session_id.sandbox.local -> homelab-sandbox-mcp:PORT
        caddyfile += f"{session_id}.{BASE_DOMAIN}:80 {{\n    reverse_proxy homelab-sandbox-mcp:{port}\n}}\n"

    # If there are no active sessions, give Caddy a default fallback to prevent 502 errors
    if not caddyfile:
        caddyfile = f":80 {{\n    respond \"Sandbox Caddy Router is online. No active sessions.\" 200\n}}\n"

    # Push to Caddy's API
    async with httpx.AsyncClient() as client:
        response = await client.post(
            CADDY_API_URL,
            content=caddyfile,
            headers={"Content-Type": "text/caddyfile"}
        )
        response.raise_for_status()
```

Finally, update the two web server tools to use this new function:

```
@mcp.tool()
async def start_web_server(session_id: str, command: str, port: int) -> str:
    """Starts a long-running web server and exposes it via the proxy."""
    safe_session = re.sub(r'^a-zA-Z0-9_-', '', session_id)
    workdir = RUNS_DIR / f"session_{safe_session}"

    if not workdir.exists():
        return "Error: Session not found."

    if safe_session in ACTIVE_SERVERS:
        ACTIVE_SERVERS[safe_session]['process'].kill()
        del ACTIVE_SERVERS[safe_session]

    # Start the process inside the MCP container
    proc = await asyncio.create_subprocess_shell(
        command,
        cwd=str(workdir),
        stdout=asyncio.subprocess.DEVNULL,
        stderr=asyncio.subprocess.DEVNULL,
        env=os.environ.copy()
    )

    # Store process and port, then update Caddy
    ACTIVE_SERVERS[safe_session] = {'process': proc, 'port': port}

    try:
        await update_caddy_routes()
    except Exception as e:
        # Rollback if Caddy update fails
        proc.kill()
        del ACTIVE_SERVERS[safe_session]
        return f"Error updating Caddy proxy: {str(e)}"

    return f"Background server started. Access it at: http://{safe_session}.{BASE_DOMAIN}"

@mcp.tool()
async def stop_web_server(session_id: str) -> str:
    """Stops the web server and removes its public route."""
    safe_session = re.sub(r'^a-zA-Z0-9_-', '', session_id)

    if safe_session in ACTIVE_SERVERS:
        ACTIVE_SERVERS[safe_session]['process'].kill()
        del ACTIVE_SERVERS[safe_session]

        try:
            await update_caddy_routes()
        except Exception as e:
            return f"Server stopped, but failed to remove route from Caddy: {str(e)}"

        return f"Server for `{safe_session}` stopped and route removed."

    return "No active server found for this session."
```

**Why this is elegant**



Because the MCP process handles the mapping, the agent never has to think about Caddy or reverse proxies. It just calls `start_web_server(session_id="my-flask-app", port=8080)`. In under a second, the MCP container spins up the Flask app on port 8080 internally, generates a Caddyfile mapping `my-flask-app.sandbox.yourdomain.com` to `homelab-sandbox-mcp:8080`, and fires it into Caddy's API.

---

User prompt: Great, please provide the content of all the required files (one by one).

Response: Here are the four files required to build this double-proxy, multi-language MCP sandbox.

Since you are running this as a persistent service inside Docker Compose alongside Caddy, the MCP server will run in **SSE (Server-Sent Events) mode** instead of `stdio`. This allows your agent to connect to it continuously over the internal Docker network via HTTP.

## 1. requirements.txt

This defines the dependencies for the MCP server itself.

```
mcp
httpx
uvicorn
starlette
```

## 2. Dockerfile

This image is based on Ubuntu 24.04 and installs all the required compilers and runtimes. It is heavily optimized to keep the layers clean.

```
FROM ubuntu:24.04
```

```
# Avoid interactive prompts during apt installations
ENV DEBIAN_FRONTEND=noninteractive
```

```
# 1. Install System Dependencies, Python, GCC, G++, and Java
```

```
RUN apt-get update && apt-get install -y \
    curl \
    wget \
    git \
    build-essential \
    python3 \
    python3-pip \
    python3-venv \
    default-jdk \
    && rm -rf /var/lib/apt/lists/*
```

```
# 2. Install Rust
```

```
ENV RUSTUP_HOME=/usr/local/rustup \
    CARGO_HOME=/usr/local/cargo \
    PATH=/usr/local/cargo/bin:$PATH
RUN curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y --no-modify-path
```

```
# 3. Install Tectonic (LaTeX)
```

```
RUN curl --proto '=https' --tlsv1.2 -fsSL https://drop-sh.fullyjustified.net | sh && \
    mv tectonic /usr/local/bin/ && \
    chmod +x /usr/local/bin/tectonic
```

```
# 4. Set up the MCP Server application
```

```
WORKDIR /app
COPY requirements.txt .
```

```
# Use a virtual environment for the MCP server dependencies to avoid PEP 668 issues
```

```
RUN python3 -m venv /opt/mcp-env
RUN /opt/mcp-env/bin/pip install --no-cache-dir -r requirements.txt
```

```
COPY server.py .
```

```
# Expose the port for the MCP SSE transport
```

```
EXPOSE 8080
```

```
# Start the MCP server using the venv Python
```

```
CMD ["/opt/mcp-env/bin/python", "server.py"]
```

## 3. docker-compose.yml

This links everything together. Notice how Traefik handles the edge ingress, passing wildcard traffic to Caddy, which then passes it to the specific ports exposed dynamically by the MCP container.

```
services:
  homelab-sandbox-caddy:
    image: caddy:latest
```

```

container_name: homelab-sandbox-caddy
restart: unless-stopped
# Expose Caddy API to the MCP container
networks:
  - proxy
  - sandbox_internal
labels:
  - "traefik.enable=true"
  # Route any subdomain ending in .sandbox.yourdomain.com to Caddy
  # Update 'yourdomain.com' to match your actual homelab domain
  - "traefik.http.routers.sandbox_caddy.rule=HostRegexp(`{subdomain:[a-zA-Z0-9-]+}.sandbox.yourdomain."
  - "traefik.http.services.sandbox_caddy.loadbalancer.server.port=80"
  # Add your Traefik TLS/resolver labels here (e.g., certresolver=letsencrypt)

homelab-sandbox-mcp:
  build: .
  container_name: homelab-sandbox-mcp
  restart: unless-stopped
  privileged: true # Required for certain process isolations and tmpfs
  tmpfs:
    - /ramdisk/sandbox:exec,mode=777
  environment:
    # Pass the base domain so the MCP server knows how to configure Caddy routes
    - SANDBOX_DOMAIN=sandbox.yourdomain.com
    - SESSION_TTL=3600
  networks:
    - sandbox_internal
  # Port 8000 is exposed to the internal network for your Agent to connect to the MCP SSE endpoint
  expose:
    - "8000"

networks:
  proxy:
    external: true
  sandbox_internal:
    driver: bridge

```

## 4. server.py

This is the complete, final version of the MCP server. It includes the background web-server tracking, Caddy API routing, multi-file execution, and session-specific Python environments.

```

import os
import sys
import uuid
import time
import shutil
import asyncio
import base64
import re
import threading
import httpx
from pathlib import Path

from mcp.server.fastmcp import FastMCP

# --- Configuration ---
SANDBOX_BASE = Path(os.environ.get("SANDBOX_DIR", "/ramdisk/sandbox"))
RUNS_DIR = SANDBOX_BASE / "runs"
BIN_DIR = SANDBOX_BASE / "bin"
VENV_DIR = SANDBOX_BASE / "venv"
SESSION_TTL_SECONDS = int(os.environ.get("SESSION_TTL", 3600))
BASE_DOMAIN = os.environ.get("SANDBOX_DOMAIN", "sandbox.local")
CADDY_API_URL = "http://homelab-sandbox-caddy:2019/load"

# Global state to track background processes and their assigned ports
# { "session_id": { "process": subprocess_object, "port": int} }
ACTIVE_SERVERS = {}

# Ensure base directories exist
RUNS_DIR.mkdir(parents=True, exist_ok=True)
BIN_DIR.mkdir(parents=True, exist_ok=True)

# --- Background Garbage Collector ---
def cleanup_stale_sessions():
    """Runs continuously in a background thread to purge expired sessions."""
    while True:
        time.sleep(300) # Check every 5 minutes
        now = time.time()

```

```

    for d in RUNS_DIR.glob("session_*"):
        if d.is_dir():
            try:
                if now - d.stat().st_mtime > SESSION_TTL_SECONDS:
                    session_id = d.name.replace("session_", "")
                    # Kill any associated background servers
                    if session_id in ACTIVE_SERVERS:
                        ACTIVE_SERVERS[session_id]['process'].kill()
                        del ACTIVE_SERVERS[session_id]
                        shutil.rmtree(d, ignore_errors=True)
            except Exception:
                pass

# Start the garbage collector as a daemon thread
threading.Thread(target=cleanup_stale_sessions, daemon=True).start()

# --- Initialize MCP Server ---
mcp = FastMCP("Homelab Execution Sandbox")

# --- Utilities ---
def get_python_path(workdir: Path | None = None) -> str:
    """Prioritizes a session-specific venv, falls back to the global one."""
    if workdir:
        session_py = workdir / ".venv" / "bin" / "python3"
        if session_py.exists():
            return str(session_py)

        global_py = VENV_DIR / "bin" / "python3"
        return str(global_py) if global_py.exists() else "/usr/bin/python3"

def get_binary(name: str) -> str:
    ram_bin = BIN_DIR / name
    return str(ram_bin) if ram_bin.exists() else (shutil.which(name) or name)

def process_file_writes(workdir: Path, code: str, entry_filename: str, files: dict[str, str] | None):
    """Safely handles file writing to disk."""
    workdir_resolved = workdir.resolve()

    if files:
        for fname, fcontent in files.items():
            target = (workdir / fname).resolve()

            # Security: Prevent Directory Traversal
            if not target.is_relative_to(workdir_resolved):
                raise ValueError(f"Path traversal blocked for file: {fname}")

            target.parent.mkdir(parents=True, exist_ok=True)

            if fcontent.startswith("data:") and ";base64," in fcontent:
                raw_b64 = fcontent.split(";base64,")[1]
                target.write_bytes(base64.b64decode(raw_b64))
            else:
                target.write_text(fcontent, encoding="utf-8")

    entry_path = workdir / entry_filename
    entry_path.write_text(code, encoding="utf-8")
    workdir.touch()

async def update_caddy_routes():
    """Rebuilds the Caddyfile in memory and pushes it instantly via the API."""
    caddyfile = ""
    for session_id, srv_info in ACTIVE_SERVERS.items():
        port = srv_info['port']
        caddyfile += f"{session_id}.{{BASE_DOMAIN}}:80 {{\n    reverse_proxy homelab-sandbox-mcp:{port}\n}}\n"

    if not caddyfile:
        caddyfile = f":80 {{\n    respond \"Sandbox Caddy Router is online. No active sessions.\" 200\n}}\n"

    async with httpx.AsyncClient() as client:
        response = await client.post(
            CADDY_API_URL,
            content=caddyfile,
            headers={"Content-Type": "text/caddyfile"}
        )
        response.raise_for_status()

# --- MCP Tools ---
@mcp.tool()
async def get_runtimes() -> str:

```

```

"""Check the availability and versions of supported compilers and runtimes."""
info = []

def check_cmd(cmd: list[str], name: str):
    try:
        import subprocess
        out = subprocess.check_output(cmd, text=True, stderr=subprocess.STDOUT).splitlines()[0]
        info.append(f"- **{name}**: {out.strip()}")
    except Exception as e:
        info.append(f"- **{name}**: Not found or error ({str(e)})")

check_cmd([get_python_path(), "--version"], "Python")
check_cmd([get_binary("tectonic"), "--version"], "LaTeX (Tectonic)")
check_cmd([get_binary("gcc"), "--version"], "C (GCC)")
check_cmd([get_binary("g++"), "--version"], "C++ (G++)")
check_cmd([get_binary("rustc"), "--version"], "Rust (rustc)")
check_cmd([get_binary("javac"), "--version"], "Java (javac)")

return "### Available Runtimes\n" + "\n".join(info)

@mcp.tool()
async def execute_code(
    language: str,
    code: str,
    files: dict[str, str] | None = None,
    timeout: int = 60,
    session_id: str | None = None
) -> str:
    """
    Executes code in a secure sandbox. Use session_id to maintain state.
    """
    lang = language.lower().strip()
    timeout = min(timeout, 300)

    is_ephemeral = not session_id
    if is_ephemeral:
        current_run_id = f"run_{int(time.time()*1000)}_{uuid.uuid4().hex[:8]}"
        workdir = RUNS_DIR / current_run_id
    else:
        safe_session = re.sub(r'^[a-zA-Z0-9_-]', '', session_id)
        if not safe_session:
            return "Error: Invalid session_id format."
        current_run_id = safe_session
        workdir = RUNS_DIR / f"session_{safe_session}"

    workdir.mkdir(parents=True, exist_ok=True)

    try:
        env = os.environ.copy()
        env["PATH"] = f"{BIN_DIR}:{VENV_DIR}/bin:{env.get('PATH', '')}"

        if lang in ["python", "py"]:
            entry_file = "script.py"
            cmd = [get_python_path(workdir), "script.py"]
        elif lang in ["latex", "tex", "tectonic"]:
            entry_file = "document.tex"
            cmd = [get_binary("tectonic"), "-o", str(workdir), "document.tex"]
        elif lang in ["c"]:
            entry_file = "main.c"
            compile_cmd = f"{get_binary('gcc')} -O2 *.c -o main -lm && ./main"
            cmd = ["/bin/sh", "-c", compile_cmd]
        elif lang in ["cpp", "c++"]:
            entry_file = "main.cpp"
            compile_cmd = f"{get_binary('g++')} -O2 -std=c++20 *.cpp -o main -lm && ./main"
            cmd = ["/bin/sh", "-c", compile_cmd]
        elif lang in ["rust", "rs"]:
            entry_file = "main.rs"
            compile_cmd = f"{get_binary('rustc')} -O main.rs -o main && ./main"
            cmd = ["/bin/sh", "-c", compile_cmd]
        elif lang in ["java"]:
            entry_file = "Main.java"
            compile_cmd = f"{get_binary('javac')} *.java && {get_binary('java')} Main"
            cmd = ["/bin/sh", "-c", compile_cmd]
        elif lang in ["sh", "bash", "shell"]:
            entry_file = "script.sh"
            cmd = ["/bin/bash", "script.sh"]
        else:
            return f"Error: Unsupported language '{lang}'"
    
```

```

try:
    await asyncio.to_thread(process_file_writes, workdir, code, entry_file, files)
except ValueError as e:
    return f"Security Error: {str(e)}"

if lang in ["sh", "bash", "shell"]:
    (workdir / entry_file).chmod(0o755)

t0 = time.time()
proc = await asyncio.create_subprocess_exec(
    *cmd,
    cwd=str(workdir),
    stdout=asyncio.subprocess.PIPE,
    stderr=asyncio.subprocess.PIPE,
    env=env
)

try:
    stdout_b, stderr_b = await asyncio.wait_for(proc.communicate(), timeout=timeout)
    exit_code = proc.returncode
except asyncio.TimeoutError:
    proc.kill()
    stdout_b, stderr_b = b"", f"Execution timed out after {timeout} seconds".encode()
    exit_code = -1

duration_ms = (time.time() - t0) * 1000

artifacts = []
for f in workdir.glob("**/*"):
    if f.is_file() and f.name not in [entry_file, "main"]:
        artifacts.append(str(f.relative_to(workdir)))

response = [
    f"### Execution Complete ({duration_ms:.0f}ms)",
    f"**Exit Code:** {exit_code}",
    f"**Session ID:** `{current_run_id}`" if not is_ephemeral else "**Run:** Ephemeral"
]

if stdout_b:
    response.append(f"\n**Stdout:**\n```\n{stdout_b.decode('utf-8', errors='replace')}\n```")
if stderr_b:
    response.append(f"\n**Stderr:**\n```\n{stderr_b.decode('utf-8', errors='replace')}\n```")

if not is_ephemeral and artifacts:
    response.append("\n**Files in Workspace:**")
    for art in artifacts:
        response.append(f"- `{art}` (Read using Resource URI: `sandbox://{current_run_id}/{art}`)")

return "\n".join(response)

finally:
    if is_ephemeral:
        await asyncio.to_thread(shutil.rmtree, workdir, ignore_errors=True)

@mcp.tool()
async def run_command(session_id: str, command: str) -> str:
    """Run an arbitrary shell command in a session (e.g., pip install)."""
    safe_session = re.sub(r'[^a-zA-Z0-9_-]', '', session_id)
    workdir = RUNS_DIR / f"session_{safe_session}"

    if not workdir.exists():
        return f"Error: Session `{safe_session}` not found."

    proc = await asyncio.create_subprocess_shell(
        command,
        cwd=str(workdir),
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE
    )

    try:
        stdout_b, stderr_b = await asyncio.wait_for(proc.communicate(), timeout=120)
        out = stdout_b.decode('utf-8', errors='replace')
        err = stderr_b.decode('utf-8', errors='replace')
        return f"Exit Code: {proc.returncode}\nStdout:\n{out}\nStderr:\n{err}"
    except asyncio.TimeoutError:
        proc.kill()
        return "Error: Command timed out after 120 seconds."

```

```

@mcp.tool()
async def start_web_server(session_id: str, command: str, port: int) -> str:
    """Starts a background web server (e.g., Flask) and exposes it via proxy."""
    safe_session = re.sub(r'^a-zA-Z0-9_-]', '', session_id)
    workdir = RUNS_DIR / f"session_{safe_session}"

    if not workdir.exists():
        return "Error: Session not found."

    if safe_session in ACTIVE_SERVERS:
        ACTIVE_SERVERS[safe_session]['process'].kill()
        del ACTIVE_SERVERS[safe_session]

    proc = await asyncio.create_subprocess_shell(
        command,
        cwd=str(workdir),
        stdout=asyncio.subprocess.DEVNULL,
        stderr=asyncio.subprocess.DEVNULL,
        env=os.environ.copy()
    )

    ACTIVE_SERVERS[safe_session] = {'process': proc, 'port': port}

    try:
        await update_caddy_routes()
    except Exception as e:
        proc.kill()
        del ACTIVE_SERVERS[safe_session]
        return f"Error updating Caddy proxy: {str(e)}"

    return f"Background server started. Access it at: http://{safe_session}.{BASE_DOMAIN}"

@mcp.tool()
async def stop_web_server(session_id: str) -> str:
    """Stops the web server and removes its public route."""
    safe_session = re.sub(r'^a-zA-Z0-9_-]', '', session_id)

    if safe_session in ACTIVE_SERVERS:
        ACTIVE_SERVERS[safe_session]['process'].kill()
        del ACTIVE_SERVERS[safe_session]

        try:
            await update_caddy_routes()
        except Exception as e:
            return f"Server stopped, but failed to remove route from Caddy: {str(e)}"

        return f"Server for `{safe_session}` stopped and route removed."

    return "No active server found for this session."

@mcp.tool()
async def delete_file(session_id: str, filename: str) -> str:
    """Deletes a specific file or directory from an active session."""
    safe_session = re.sub(r'^a-zA-Z0-9_-]', '', session_id)
    workdir = RUNS_DIR / f"session_{safe_session}"

    if not workdir.exists():
        return f"Error: Session `{safe_session}` not found."

    target = (workdir / filename).resolve()

    if not target.is_relative_to(workdir.resolve()):
        return f"Security Error: Path traversal blocked."

    if target == workdir.resolve():
        return "Error: Cannot delete the root directory."

    if not target.exists():
        return f"Error: Path `{filename}` does not exist."

    def _delete():
        if target.is_dir():
            shutil.rmtree(target)
        else:
            target.unlink()

    try:
        await asyncio.to_thread(_delete)
        return f"Successfully deleted `{filename}`."

```

```

except Exception as e:
    return f"Error deleting `{filename}`: {str(e)}"

@mcp.tool()
async def close_session(session_id: str) -> str:
    """Manually close and purge a session to free memory."""
    safe_session = re.sub(r'^a-zA-Z0-9_', '', session_id)
    workdir = RUNS_DIR / f"session_{safe_session}"

    if safe_session in ACTIVE_SERVERS:
        ACTIVE_SERVERS[safe_session]['process'].kill()
        del ACTIVE_SERVERS[safe_session]
        await update_caddy_routes()

    if workdir.exists():
        await asyncio.to_thread(shutil.rmtree, workdir, ignore_errors=True)
        return f"Session `{safe_session}` closed and purged."
    return f"Session `{safe_session}` not found."

# --- MCP Resources ---
@mcp.resource("sandbox://{session_id}/{filename}")
def read_sandbox_file(session_id: str, filename: str) -> str | bytes:
    """Read the contents of a generated file from an active session."""
    safe_session = re.sub(r'^a-zA-Z0-9_', '', session_id)
    workdir = RUNS_DIR / f"session_{safe_session}"

    if not workdir.exists():
        raise ValueError("Session not found.")

    target = (workdir / filename).resolve()

    if not target.is_relative_to(workdir.resolve()):
        raise ValueError("Path traversal blocked.")

    if not target.exists() or not target.is_file():
        raise ValueError("File not found.")

    ext = target.suffix.lower()
    if ext in [".pdf", ".png", ".jpg", ".jpeg", ".zip", ".tar", ".gz"]:
        return target.read_bytes()

    return target.read_text(encoding="utf-8", errors="replace")

if __name__ == "__main__":
    # Start the MCP server using SSE transport for continuous Docker operation
    mcp.run(transport='sse', host='0.0.0.0', port=8000)

```

---

*User prompt: How do I configure my agent (like Open-WebUI or Hermes) to connect to this new SSE-based MCP server over the network?*

Response: Because your MCP server is now running in **SSE (Server-Sent Events) mode**, it behaves like a standard REST API rather than a raw command-line process. This makes it incredibly easy to connect over your homelab network.

The golden rule for SSE connections is that you must point the agent to the `/sse` endpoint, not just the root domain.

Assuming your agent (Open-WebUI or Hermes) is running on the same Docker host, here is how you connect them.

## The Connection URL

If your agent is attached to the same Docker network (which we named `sandbox_internal` in the `docker-compose.yml`), it can resolve the container name directly: `http://homelab-sandbox-mcp:8000/sse`

*(If your agent is running outside of Docker on the host machine, you would expose port 8000 in the compose file and connect to `http://localhost:8000/sse` or `http://<YOUR_NAS_IP>:8000/sse`)*

## Option 1: Configuring Open-WebUI

Open-WebUI has native support for connecting to remote MCP servers.

1. **Ensure Network Access:** Make sure your Open-WebUI container is attached to the `sandbox_internal` network. (You can add `sandbox_internal` to the `networks` block of your Open-WebUI `docker-compose.yml`).
2. **Navigate to MCP Settings:** In Open-WebUI, log in as an administrator. Go to **Admin Panel > Settings > MCP Servers**.
3. **Add the Server:**
  - **Name:** Homelab Sandbox
  - **Transport Type:** Select **SSE** (do *not* select stdio/command).

- **SSE URL:** Enter `http://homelab-sandbox-mcp:8000/sse`

4. **Save and Connect:** Open-WebUI will immediately reach out to the `/sse` endpoint, read all the tools (`execute_code`, `start_web_server`, etc.), and make them available to your LLM chats.

## Option 2: Configuring a Custom Python Agent (Hermes)

If your Hermes agent is a custom Python script using frameworks like LangChain, LlamaIndex, or the raw MCP SDK, you need to use the `sse_client` transport instead of `stdio_client`.

Here is the exact code snippet to connect your agent to the sandbox programmatically:

```
import asyncio
from mcp import ClientSession
from mcp.client.sse import sse_client

async def run_agent():
    # The URL of your Dockerized MCP server
    server_url = "http://homelab-sandbox-mcp:8000/sse"

    # Initialize the SSE connection
    async with sse_client(server_url) as (read_stream, write_stream):
        async with ClientSession(read_stream, write_stream) as session:

            # Initialize the handshake
            await session.initialize()

            # Verify the tools are loaded
            tools = await session.list_tools()
            print("Loaded Tools:", [t.name for t in tools.tools])

            # Example: Instruct the agent to execute code
            result = await session.call_tool(
                "execute_code",
                arguments={
                    "language": "python",
                    "code": "print('Hello from the remote sandbox!')"
                }
            )
            print(result.content[0].text)

if __name__ == "__main__":
    asyncio.run(run_agent())
```

---

*User prompt: How do I configure Open-WebUI or my agent so that when I attach a file in the chat interface, it automatically gets passed into the files dictionary of the MCP execute\_code tool?*

Response: The most important concept to understand here is that **the UI does not talk directly to the MCP tool**. The LLM sits in the middle.

When you attach a file in a chat interface, the UI extracts that file and feeds it to the LLM as context. It is the LLM's job to realize, *"Ah, the user wants me to run code against this data, so I need to copy this data into the files parameter of the execute\_code tool."*

Because the LLM is the bridge, you simply have to instruct it on how to handle attachments. Here is how to do this in both Open-WebUI and a custom Python agent.

## Option 1: Configuring Open-WebUI (The System Prompt Method)

When you upload a text-based file (like a CSV, JSON, or Python script) to Open-WebUI, it automatically extracts the text and injects it into the prompt behind the scenes. You just need a strong System Prompt to tell the LLM what to do with that injected text.

### 1 Create a Custom Model for the Sandbox

Keeps your main chat clean

In Open-WebUI, go to **Workspace -> Models** and click the **+** button to create a new model. Base it on your smartest available LLM (like GPT-4o, Claude 3.5 Sonnet, or Llama 3).

### 2 Write the Routing Instructions

In the **System Prompt** field, paste the following instructions to force the LLM to map uploaded documents to your MCP tool:

"You are an expert developer connected to a secure code execution sandbox. When the user attaches files or documents to the chat and asks you to process them, do NOT try to write a script that reads from the local file system.

Instead, you must copy the raw content of the attached files and pass them directly into the `files` dictionary argument of the `execute_code` tool. Use the document's filename as the dictionary key, and the document's content as the value."

### 3 Enable the MCP Tool



Scroll down in the model settings and ensure the Homelab Sandbox MCP server you configured earlier is toggled ON for this specific model. Save the model.

Now, when you use this custom model, attach a file named `data.csv`, and type *"Write a python script to plot this"*, the LLM will automatically generate an `execute_code` call with files: `{"data.csv": "col1,col2\n1,2..."}`.

## Option 2: Custom Python Agent (The Base64 Injection Method)

If you are building your own Hermes agent in Python, relying on the LLM to copy-paste massive files into the tool call can exhaust its output token limit or cause hallucinations.

Instead, you should intercept the file upload in your Python code, format it as a base64 Data URI (which we specifically built your sandbox to support!), and pass it to the agent as a pre-formatted tool argument.

Here is how you handle file attachments programmatically before the LLM even sees them:

```
import base64
import mimetypes
from pathlib import Path

def prepare_file_for_sandbox(filepath: str) -> dict:
    """
    Reads a local file (text or binary like an image/PDF) and
    formats it for the MCP sandbox's `files` dictionary.
    """
    path = Path(filepath)
    mime_type, _ = mimetypes.guess_type(path)
    mime_type = mime_type or "application/octet-stream"

    # Read the file as raw bytes
    raw_bytes = path.read_bytes()

    # Encode to base64
    b64_string = base64.b64encode(raw_bytes).decode('utf-8')

    # Create the Data URI format that our server.py expects
    data_uri = f"data:{mime_type};base64,{b64_string}"

    return {
        "filename": path.name,
        "content": data_uri
    }

# --- Inside your Agent's execution flow ---

# 1. User uploads an image for OpenCV processing
uploaded_image = prepare_file_for_sandbox("./local_uploads/test_image.jpg")

# 2. You artificially inject this into the LLM's tool call, or pass it
# as part of the context so the LLM knows it has access to it.
files_payload = {
    uploaded_image["filename"]: uploaded_image["content"]
}

# The LLM now just generates the code, and your backend merges the files_payload
# into the tool arguments before sending it to the MCP server.
```

By using the base64 method in a custom agent, you can pass heavy binary files (images, audio files for speech processing datasets, compiled libraries) straight into the sandbox without forcing the LLM to "read" the raw binary data.

---

*User prompt: How to register and use this MCP on hermes agents ?*

Response: Hermes Agent has native, built-in support for remote HTTP/SSE MCP servers, making registration incredibly straightforward. You don't need to write any custom integration code; you just need to point Hermes to the URL.

Here is how to connect them.

## Step 1: Expose the Port (If Needed)

If your Hermes agent is running natively on your host machine (e.g., via the standard install script in `~/hermes`), it won't automatically be able to see inside the `sandbox_internal` Docker network.

In the `docker-compose.yml` file, change expose to ports so the host can reach it:

```
homelab-sandbox-mcp:
  # ...
  ports:
    - "8000:8000"
```

## Step 2: Edit the Hermes Config

Open your Hermes configuration file at `~/hermes/config.yaml` and add the sandbox to the `mcp_servers` block.

```
mcp_servers:
  homelab_sandbox:
    url: "http://127.0.0.1:8000/sse"
```

*(If Hermes is running on a different Proxmox VM or LXC container than the sandbox, replace `127.0.0.1` with the IP of your Docker host).*

## Step 3: Start Chatting

Hermes handles the rest automatically. When you launch `hermes chat` (or open the Hermes WebUI / Desktop App), it reaches out to the `/sse` endpoint during startup, negotiates the capabilities, and registers all your new tools (`execute_code`, `start_web_server`, `delete_file`) seamlessly.

Because Hermes Agent shares the same core config across all its interfaces, the sandbox will immediately be available whether you are using the terminal, the WebUI, or the messaging gateway.

This unlocks incredibly powerful workflows for your current projects:

- **Speech Processing Research:** You can drop a raw audio file into the chat and ask Hermes to apply `cmpPNGC` feature extraction. It will write the Python script, execute it in the sandbox, and return the processed output.
- **Educational Materials:** You can say, *"Generate a math revision sheet for Mariam using XeLaTeX and TikZ."* Hermes will write the `.tex` file, use the sandbox to run Tectonic, and provide the compiled PDF back to you.

## Sources

1. [Hermes Agent — Open-Source AI Agent with Persistent Memory](#)
  2. [MCP \(Model Context Protocol\) | Hermes Agent - NOUS RESEARCH](#)
  3. [Hermes WebUI: The best way to use Hermes Agent from the web or from your phone! - GitHub](#)
-